Product Name: TaskFlask
Team: The Goatz
Date: 07/21/2026

Test Plan and Report

User story: As a user, I want to create and manage projects so that my team's work is organized.

Scenario 1: Create and open a project (Pass)

- **Input:**
  - Project name = TaskFlask Demo
  - Project description = ## Goal followed by Build a clear Scrum workspace.
- **Procedure:**
  - Open the Projects page and select New Project.
  - Enter the project name and description.
  - Select Create Project.
  - Open the newly created project.
- **Expected Output:**
  - The app displays the message Project created.
  - TaskFlask Demo appears on the Projects page and opens successfully.
  - The Markdown heading in the description renders as formatted content rather than raw ## text.

Scenario 2: Edit an existing project (Pass)

- **Input:**
  - Existing project = TaskFlask Demo
  - Updated name = TaskFlask Final Demo
  - Updated description = A polished project for the final review.
- **Procedure:**
  - Open TaskFlask Demo.
  - Select Edit Project.
  - Replace the name and description with the updated values.
  - Submit the form.
- **Expected Output:**
  - The app displays Project updated.
  - The project detail page shows TaskFlask Final Demo and the new description.
  - The project's updated timestamp changes and is displayed in Pacific time.

Scenario 3: Reject an invalid project and delete a valid project (Pass)

- **Input:**
  - Invalid project name = blank
  - Valid project to delete = Temporary Project
- **Procedure:**
  - Open New Project, leave the name blank, and submit.
  - Confirm that the invalid project is not created.
  - Create Temporary Project with a valid name.
  - Use the project's Delete action and confirm the browser prompt.

- **Expected Output:**
  - The blank submission displays Project name is required.
  - Deleting the valid project displays a confirmation message containing deleted.
  - Temporary Project disappears from the list; if no projects remain, the No projects yet empty state is displayed.

User story: As a user, I want to create and manage sprints inside a project so that work is divided into planning cycles.

Scenario 1: Create and edit a sprint (Pass)

- **Input:**
  - Project = TaskFlask Final Demo
  - Sprint name = Foundation Sprint
  - Goal = **Finish** the application foundation.
  - Start date = 2026-07-13
  - End date = 2026-07-17
  - Updated sprint name = Project Foundation
- **Procedure:**
  - Open the project and select New Sprint.
  - Enter the sprint name, Markdown goal, start date, and end date.
  - Select Create Sprint.
  - Open Edit for the sprint, change its name to Project Foundation, and submit.
- **Expected Output:**
  - The app displays Sprint created. and later Sprint updated.
  - The project page shows Project Foundation with the correct dates and Active status.
  - The word Finish renders with Markdown emphasis in the sprint goal.

Scenario 2: Reject invalid sprint values (Pass)

- **Input:**
  - Sprint name = blank
  - Invalid date range = start 2026-07-20 and end 2026-07-15
- **Procedure:**
  - Submit the sprint form with a blank name.
  - Submit again with a valid name but the end date before the start date.
- **Expected Output:**
  - The blank name displays Sprint name is required.
  - The invalid range displays End date should be on or after the start date.
  - Neither invalid sprint is saved.

Scenario 3: Complete, reopen, and delete a sprint (Pass)

- **Input:**
  - Existing sprint = Project Foundation with Active status
- **Procedure:**
  - Open the sprint board and select Mark Complete.
  - Return to the sprint and select Reopen.
  - Use Delete Sprint and confirm the prompt.

- **Expected Output:**
  - Completing displays Sprint marked completed., changes status to Completed, and records completed_at.
  - Reopening displays Sprint reopened., changes status to Active, and clears completed_at.
  - Deleting displays a message containing deleted and removes the sprint from its project.

User story: As a user, I want complete task records so that sprint work can be estimated, assigned, scheduled, and reviewed.

Scenario 1: Create, edit, and delete a task (Pass)

- **Input:**
  - Title = Create database schema
  - Description = ## Acceptance Criteria followed by - Create all required tables
  - Status = To Do
  - Priority = High
  - Story points = 20
  - Assignee = Team member 1
  - Added on = 2026-07-13
  - Due date = 2026-07-17
- **Procedure:**
  - Open the sprint board and select New Task.
  - Enter all task values and submit.
  - Edit the task: change the title to Create final database schema, priority to Medium, and assignee to Team member 2.
  - Delete the edited task and confirm the prompt.
- **Expected Output:**
  - The task card initially displays High, 20 SP, Team member 1, Added 2026-07-13, and Due 2026-07-17.
  - The description renders the Markdown heading and list safely.
  - The edited values persist and Task updated. is displayed.
  - Deletion displays Task deleted. and removes the task card.

Scenario 2: Reject invalid task story points and dates (Pass)

- **Input:**
  - Title = Estimate validation
  - Story-points test values = blank, 2.5, 0, and 20
  - Invalid added-on date = not-a-date
- **Procedure:**
  - Submit the task with story points blank.
  - Submit with story points 2.5.
  - Submit with story points 0.
  - Submit with 5 story points and added-on date not-a-date.
  - Submit with 20 story points and a valid added-on date.
- **Expected Output:**
  - Blank and 2.5 display Story points are required and must be a whole number.
  - Zero displays Story points must be at least 1.

    - The invalid date displays Added on date must use YYYY-MM-DD.
    - The valid 20-point task is saved and displays 20 SP, confirming that estimates are not limited to Fibonacci values.

Scenario 3: Create and edit a historical completion date (Pass)

- **Input:**
    - Task title = Historical task
    - Status = Done
    - Story points = 8
    - Added on = 2026-07-10
    - Completed on = 2026-07-12, then update to 2026-07-14
- **Procedure:**
    - Create the Done task with the first completion date.
    - Open Edit Task and change Completed On to 2026-07-14.
    - Submit the edit and reopen the form.
- **Expected Output:**
    - The task is stored as Done with the first completion date.
    - The edited completion date persists as 2026-07-14 and is used by the burnup calculation.
    - The task card and edit form show the saved completion date.

Scenario 4: Reject invalid completion dates and clear the date when reopening (Pass)

- **Input:**
    - Added on = 2026-07-10
    - Invalid completed on = 2026-07-09
    - Valid completed on = 2026-07-12
- **Procedure:**
    - Submit a Done task with completion before its added-on date.
    - Correct the completion date to 2026-07-12 and save.
    - Edit the task status from Done to In Progress and submit.
- **Expected Output:**
    - The invalid date displays Completed on date cannot be before the added on date.
    - The corrected Done task saves successfully.
    - Reopening the task clears completed_at so unfinished work is not counted as completed story points.

User story: As a user, I want a visual three-column board so that I can update and understand sprint status quickly.

Scenario 1: Drag a task between To Do, In Progress, and Done (Pass)

- **Input:**
    - Task = Drag me; initial status = To Do
- **Procedure:**
    - Open the sprint board and locate Drag me in To Do.
    - Drag only the task card to In Progress.
    - Drag the card again to Done.
    - Reload the page.

- **Expected Output:**
  - The board displays To Do, In Progress, and Done columns with accurate counts.
  - Only the card uses the drag preview; the entire column does not appear to move.
  - Each move displays Task status updated. and persists after reload.
  - No fallback Move form appears on the task card.

Scenario 2: Preserve card order within and between columns (Pass)

- **Input:**
  - Done cards = hbguh followed by efeffe
  - Incoming To Do card = Incoming
- **Procedure:**
  - Drag hbguh above efeffe in Done.
  - Drag Incoming between hbguh and efeffe.
  - Reload the sprint board.
- **Expected Output:**
  - The Done order remains hbguh, Incoming, efeffe.
  - Stored board_order values are 0, 1, and 2.
  - The card that was created earlier does not automatically jump back to the top.

Scenario 3: Filter tasks by assignee, priority, and status (Pass)

- **Input:**
  - Task = Updated task
  - Assignee = Jordan
  - Priority = Medium
  - Status = Done
- **Procedure:**
  - Open the sprint board.
  - Choose Jordan, Medium, and Done in the three filter controls.
  - Select Apply.
  - Select Clear after verifying the result.
- **Expected Output:**
  - Only tasks matching all three filters appear.
  - Updated task remains visible and the page displays Filters active.
  - Clear restores the unfiltered board.
  - Unknown priority or status query values are ignored safely.

User story: As a team, we want progress to use story points so that completed effort is measured more accurately than task count.

Scenario 1: Calculate sprint story-point progress (Pass)

- **Input:**
  - To Do task = 2 SP
  - In Progress task = 3 SP
  - Done task = 5 SP
- **Procedure:**
  - Create the three tasks in one sprint with the listed statuses and points.

    - Open the sprint detail page.
    - Compare the displayed totals with the stored task data.
- **Expected Output:**
    - The sprint reports 10 total SP and 5 completed SP.
    - Progress displays 50%, even though only one of three tasks is Done.
    - Task counts still report one To Do, one In Progress, and one Done for board context.

Scenario 2: Calculate project story-point progress consistently (Pass)

- **Input:**
    - Done task = 8 SP
    - To Do task = 13 SP
- **Procedure:**
    - Create both tasks in the same project.
    - Open the dashboard, Projects page, and project detail page.
- **Expected Output:**
    - All three pages report 8 completed SP of 21 total SP.
    - The calculated percentage is 38% after integer rounding.
    - The same database-backed values appear consistently across views.

User story: As a team, we want one cumulative project burnup chart so that progress across every sprint is visible over time.

Scenario 1: Combine scope and completion from multiple sprints (Pass)

- **Input:**
    - Project = Unified Project
    - Sprint 1 task = 3 SP, added 2026-07-01, completed 2026-07-02
    - Sprint 2 task = 5 SP, added 2026-07-03, still To Do
    - Chart end date = 2026-07-04
- **Procedure:**
    - Open the project detail page.
    - Locate Project burnup below Planning cycles.
    - Compare each daily point with the supplied task dates.
- **Expected Output:**
    - The chart states that it covers all 2 sprints and does not provide a per-sprint selector.
    - Total scope by day is 3, 3, 8, 8.
    - Completed points by day are 0, 3, 3, 3.
    - The summary reports 3 completed SP, 8 total SP, and 38%.

Scenario 2: Use added-on and completed-on dates instead of due dates (Pass)

- **Input:**
    - Task = Build application factory
    - Added on = 2026-07-13
    - Due date = 2026-07-17
    - Completed on = 2026-07-20
    - Story points = 3
- **Procedure:**

    - Create the task with the listed historical dates.
    - Open the project burnup chart.
    - Inspect where total scope and completed work increase.
- **Expected Output:**
    - Total scope increases on July 13 when the work enters the project.
    - Completed points increase on July 20 when the work is completed.
    - The due date July 17 does not cause the completed line to rise.

Scenario 3: Display the burnup empty state without stale chart data (Pass)

- **Input:**
    - Project with at least one sprint and zero task story points
- **Procedure:**
    - Open the project detail page before creating tasks.
    - Confirm the burnup summary and chart area.
    - Add a task, verify the chart, delete the task, and reload.
- **Expected Output:**
    - With no chartable scope, the page shows No story points to chart yet.
    - No old line, legend, or points remain visible after the task is removed.
    - After valid tasks are added, the chart renders only the current project's cumulative data.

User story: As a user, I want clear navigation, safe Markdown, useful empty states, and accurate dashboard information so that the app is easy to understand.

Scenario 1: Render safe Markdown in project, sprint, and task text (Pass)

- **Input:**
    - Markdown examples = headings, bold text, links, lists, checklists, quotes, and code
    - Unsafe examples = raw script HTML and javascript: links
- **Procedure:**
    - Create a project description, sprint goal, and task description using the supported Markdown examples.
    - Include raw HTML and an unsafe link in a separate test value.
    - Open the project list, project detail, and sprint board.
- **Expected Output:**
    - Supported Markdown renders as headings, emphasis, links, lists, checklists, quotes, and code.
    - Raw HTML is escaped and unsafe links are rejected.
    - The same safe rendering behavior appears in project descriptions, sprint goals, and task descriptions.

Scenario 2: Use navigation, task previews, and empty states (Pass)

- **Input:**
    - Project contains Foundation Sprint and Polish Sprint
    - Foundation task = Create schema, 5 SP
    - Polish task = Refine board, 8 SP
- **Procedure:**
    - Open Projects and expand View tasks for the project.
    - Confirm each task's sprint tag and story-point value.

    - Open the project, use Back to Projects, open a sprint, and use Back to Project.
    - Remove all records and review empty project, sprint, task, and chart states.
- **Expected Output:**
    - Each previewed task is tagged with the correct sprint and SP value.
    - Back links return to the correct parent page.
    - Empty states explain the next available action without duplicating primary create buttons.

Scenario 3: Verify dashboard sorting, preview limits, counts, and Pacific time (Pass)

- **Input:**
    - Four projects, including Alpha Project and Zulu Project
    - Distinct assignees = Alex and Jordan; duplicate Alex and one blank assignee
    - Sort values = name and not-a-real-sort
    - UTC timestamp = 2026-07-14T18:14:00+00:00
- **Procedure:**
    - Open the dashboard with name sorting and verify alphabetical order.
    - Open it with the invalid sort value.
    - Count visible project previews and open the full Projects link.
    - Compare dashboard counts and the converted timestamp with the input data.
- **Expected Output:**
    - Alpha Project appears before Zulu Project.
    - An unknown sort falls back safely to Recently updated.
    - The dashboard shows at most three previews and provides View all 4 projects; the full page shows all projects.
    - Team Members reports 2 distinct nonblank assignees.
    - The timestamp displays Jul 14, 2026 at 11:14 AM PDT.

User story: As a developer, I want the database commands and automated test suite to verify the release so defects are found before the project review.

Scenario 1: Initialize and launch TaskFlask (Pass)

- **Input:**
    - Command = python -m flask --app run init-db
    - Launch command = python run.py
    - URL = http://127.0.0.1:5000/
- **Procedure:**
    - Activate the project virtual environment and install requirements.
    - Run init-db.
    - Start the Flask app and open the local URL.
- **Expected Output:**
    - The command initializes projects, sprints, tasks, and activity_log tables.
    - The dashboard returns HTTP 200 and displays TaskFlask.
    - The shared style.css and app.js assets load successfully.

Scenario 2: Migrate an older database without losing task data (Pass)

- **Input:**
    - Older task = Existing task with 13 SP and created date 2026-07-15

    - Command = python -m flask --app run migrate-db
    - Updated estimate after migration = 20 SP
- **Procedure:**
    - Create the older task schema and insert Existing task.
    - Run migrate-db.
    - Change Existing task from 13 SP to 20 SP and reload it.
- **Expected Output:**
    - The command reports Migrated the TaskFlask database.
    - Existing task is preserved and its added_on date is backfilled as 2026-07-15.
    - The migrated schema accepts the positive whole-number estimate 20.

Scenario 3: Run the complete automated test suite (Pass)

- **Input:**
    - Command = python -m pytest -q
    - Repository root = TaskFlasktester
- **Procedure:**
    - Activate the virtual environment.
    - Run the test command from the repository root.
    - Review the final progress line and failure summary.
- **Expected Output:**
    - The console displays 46 test progress dots followed by [100%].
    - The final result is 46 passed, 0 failed, and 0 errors.
    - Any future failure, error, or collection problem blocks release approval until reviewed.

Automated Test Report

- **Test command:** python -m pytest -q
- **Result:** 46 passed, 0 failed, 0 errors in 0.91 seconds.
- **Executed:** July 21, 2026 from the TaskFlasktester repository root.
- **Test directories and files:**
    - tests/test_app.py — Pacific-time formatting
    - tests/test_projects.py — app setup, database commands, project workflows, dashboard, previews, sorting, and counts
    - tests/test_sprints.py — sprint CRUD, validation, navigation, completion, and reopening
    - tests/test_tasks.py — task CRUD, validation, completion dates, drag-only board behavior, filtering, and persistent ordering
    - tests/test_progress.py — story-point progress calculations
    - tests/test_burnup.py — cumulative project burnup calculations and rendering
    - tests/test_markdown.py — safe Markdown rendering and sanitization
- **Release assessment:**
    - All documented scenarios passed for the current classroom release build.
    - No automated release-blocking defect is currently known.
    - Before the live demonstration, perform one final browser smoke test of drag-and-drop, hover actions, responsive layout, and prepared demonstration data.